Auditoria da Interface de Monitoramento Veicular e Agentes de IA

Sistema: AITrack + DataDrivr
Data da auditoria: 05/11/2026
Aplicação auditada: `aitrackdatadrivr/frontend`
Ambiente: frontend em `http://localhost:3000`, API em `http://localhost:5009`, receptor Tracker em `9000`
Fontes monitoradas: Tracker e banco Dirija Bem
Método: navegação real com Playwright, inspeção visual, captura das chamadas HTTP e leitura do frontend e backend ativos.

---

1. Resumo executivo

A aplicação já oferece uma boa prova de conceito visual: mapa ao vivo, seleção de veículos Tracker, seleção de usuários Dirija Bem, placar comportamental, timeline, análises, lista de monitores, alertas e catálogo de eventos. O backend recebe posições Tracker, consulta dados Dirija Bem e calcula quatro tipos de evento comportamental.

Entretanto, a interface ainda **não é uma central de monitoramento operacional**. Ela mistura dados reais e dados demonstrativos sem avisar o operador. Alertas, catálogo de eventos e parte do detalhe dos monitores parecem funcionais, mas são mocks. Também não existem gestão de cercas, sensores, incidentes, contatos, comunicações, escalonamentos, comandos ao dispositivo ou execução autônoma de agentes.

O maior problema não é estético. É a falta de uma cadeia operacional completa:

> telemetria → detecção → correlação → incidente → decisão → ação → confirmação → acompanhamento → encerramento auditável.

Hoje existem os primeiros dois elementos dessa cadeia e telas que simulam parte dos demais. O próximo norte deve ser transformar a aplicação em um **centro de comando orientado a incidentes**, no qual os agentes de IA executem trabalho semelhante ao de um monitor humano, mas sempre dentro de políticas explícitas.

Veredito por área

Área	Situação atual	Maturidade
Recepção Tracker	Dados reais, simulador e socket ativos	Parcial
Integração Dirija Bem	Consulta de usuários/rotas e simulador direto no banco	Parcial
Mapa ao vivo	Funciona para posições e trilhas básicas	Parcial
Eventos comportamentais	4 detectores reais	Parcial
Timeline	Real, porém sem agregação e filtros	Parcial
Análises	Reais, porém limitadas e distorcidas pelo excesso de eventos	Parcial
Monitores	Cadastro/API existem; detalhe mistura API e mocks	Protótipo
Alertas	Tela inteiramente mock	Demonstração
Catálogo de eventos	Tela inteiramente mock	Demonstração
Cercas, rotas e áreas	Não implementados	Ausente
Sensores e comandos	Não implementados	Ausente
Incidentes	Não implementados	Ausente
Agentes autônomos	Não existe runtime de IA nem executor de ações	Ausente
Comunicação com proprietário	Não implementada	Ausente
Auditoria, acesso e governança	Não implementados	Ausente

---

2. Como a auditoria foi realizada

Foram percorridas 11 situações de interface em navegador real:

1. Dashboard inicial;
2. Dashboard com veículo Tracker selecionado;
3. Timeline;
4. Análises;
5. Lista de Monitores AI;
6. Detalhe do Monitor #1;
7. Filtro de veículo dentro do monitor;
8. Lista de Alertas;
9. Detalhe de um alerta;
10. Catálogo de Eventos;
11. Filtro de eventos críticos.

Durante a navegação foram observadas 39 respostas de API, todas HTTP 200. Não houve erro de página, falha de requisição ou resposta HTTP inválida. Isso prova estabilidade básica do fluxo examinado, mas não prova que as informações são reais: várias telas são preenchidas diretamente por arquivos de mock e, portanto, não fazem requisições.

A seleção de um usuário Dirija Bem não foi acionada durante a auditoria porque o componente executa um `POST` que inicia replay. A auditoria foi mantida sem mutações intencionais.

APIs realmente observadas pela interface

- `/api/dirijabem/users`
- `/api/fleet/events`
- `/api/fleet/scores`
- `/api/fleet/stats`
- `/api/monitors`
- `/api/monitors/1`
- `/api/monitors/1/vehicles`
- `/api/posicoes`

Não foram observadas chamadas reais para alertas, catálogo de eventos, ações, cercas, sensores, incidentes, comunicações ou estado de agentes.

---

3. Inventário do que existe hoje

3.1 Navegação principal

A barra superior possui seis opções:

- Dashboard;
- Timeline;
- Análises;
- Monitores AI;
- Alertas;
- Eventos.

O layout é desktop, com menu superior, painel lateral de aproximadamente 350 px e mapa ocupando o restante. É adequado para demonstração em monitor grande, mas ainda não possui navegação por domínio, busca global, central de notificações, identificação do operador, empresa/tenant, perfil, configurações ou ajuda operacional.

3.2 Dashboard

O que existe e usa dados reais

- mapa Leaflet/OpenStreetMap;
- posições recentes dos veículos Tracker;
- seleção entre 10 dispositivos simulados `SIM-1000` a `SIM-1009`;
- seleção de usuários Dirija Bem;
- score médio;

- total de eventos do dia;
- quantidade de veículos;
- melhores e piores scores;
- distribuição dos scores;
- marcadores e trilha do veículo selecionado;
- rota e último ponto para usuários Dirija Bem.

Problemas encontrados

- os dois mundos são apresentados como seletores separados, e não como uma frota unificada;
- não há placa, modelo, grupo, cliente, motorista atual, origem do dado ou estado operacional no seletor;
- não há busca, filtro, favoritos ou agrupamento;
- não há indicação de última comunicação, atraso, sinal GPS/GSM, ignição, alimentação, bateria ou disponibilidade;
- não há legenda nem controle de camadas;
- não há modo de tela cheia, agrupamento de marcadores ou foco por incidente;
- dados pessoais do Dirija Bem aparecem diretamente, sem controles visíveis de acesso;
- selecionar um usuário Dirija Bem inicia processamento por `POST`, um efeito colateral inesperado para uma simples seleção;
- a interface não distingue dado vivo, replay e simulação;
- não existem cercas, rotas autorizadas, zonas de risco ou incidentes desenhados no mapa.

3.3 Timeline

O que existe

- lista real de eventos recentes;
- tipo, veículo, horário e severidade;
- atualização periódica.

Problema operacional crítico

Durante a auditoria, a tela mostrou vários eventos de excesso de velocidade para o mesmo veículo em intervalos de aproximadamente cinco segundos. O dashboard chegou a exibir 2.256 eventos no dia. Isso transforma uma única condição contínua em dezenas ou centenas de registros e derruba scores para zero.

Antes de automatizar contatos, os eventos precisam ser agrupados em episódios. Um episódio de velocidade deve ter início, pico, duração e término; não deve gerar uma nova notificação a cada posição.

O que falta

- filtros por período, origem, frota, veículo, motorista, tipo, severidade e estado;
- busca e paginação/virtualização;
- localização e contexto do evento;
- valor medido, limite aplicado e evidências;
- agrupamento em episódios;
- vínculo com incidente e alerta;
- marcar como falso positivo;
- assumir, reconhecer, comentar e escalar;
- exportar e compartilhar.

3.4 Análises

O que existe e usa dados reais

- distribuição de eventos;
- ranking de veículos;
- score por veículo;
- correlação simples entre score e eventos.

O que falta

- seletor de período e comparação com período anterior;
- separação Tracker/Dirija Bem;
- tendências por motorista, veículo, frota, região e horário;
- métricas específicas do Dirija Bem;
- explicação auditável do score;
- análise de viagens, distância e exposição ao risco;
- normalização por quilômetros ou horas dirigidas;
- drill-down do gráfico ao evento e à posição;
- relatórios agendados e exportação;
- indicadores operacionais de incidentes e atendimento.

Enquanto o sistema contar uma condição persistente como centenas de eventos, os gráficos e scores não serão confiáveis para gestão.

3.5 Monitores AI

O que existe

- API e lista real de cinco monitores;
- quatro aparecem ativos;
- nome, descrição, intervalo e quantidade de veículos;
- vínculo de veículos com monitor;
- tela de detalhe e filtro por veículo.

O que parece real, mas não é

- o `App.tsx` entrega `mockVehiclesInMonitors` ao detalhe;
- o `MonitorDetailView.tsx` usa `mockEvents`;
- eventos como colisão, pânico, cerca, violação de rastreador e desperdício de combustível aparecem sem terem sido detectados;
- o detalhe pode mostrar score 85 e zero eventos como fallback, ocultando falha de integração;
- os eventos exibidos podem pertencer a veículos que nem fazem parte do monitor selecionado.

O que falta para ser um agente

- criar, editar, duplicar, pausar e excluir;
- definir missão e resultado esperado;
- definir frota, veículos, motoristas, áreas e horários sob responsabilidade;
- escolher detectores e parâmetros;
- configurar autonomia e ações permitidas;

- configurar contatos, canais e escalonamento;
- estado: saudável, degradado, pausado, aguardando aprovação ou falha;
- última execução, próxima execução, latência e backlog;
- histórico de decisões e justificativas;
- ferramentas disponíveis ao agente;
- memória operacional e contexto do cliente;
- modelo de IA, versão do prompt, consumo e custo;
- simulação, teste e publicação de uma nova versão;
- possibilidade de intervenção humana.

3.6 Alertas

A tela possui uma apresentação visual útil: contadores, filtros, severidade, estado, descrição, bloco de "análise da IA", recomendações e detalhe. Entretanto, todo o conteúdo vem de `mockAlerts` e `mockAlertStats`.

Os botões de reconhecer e resolver apenas abrem uma mensagem no navegador com o texto "Mock - não persiste". Não foi feita qualquer chamada de API. A interface chega a afirmar contato com motorista e confirmação por câmera, sem existir integração correspondente. Em operação real isso seria uma falsa evidência.

O que falta

- alertas persistentes vindos do backend;
- estados e transições válidas;
- responsável e fila de atendimento;
- evidências e eventos relacionados;
- ações executadas e respectivos resultados;
- comunicação enviada, entregue, lida e respondida;
- chamadas iniciadas, atendidas e gravadas conforme política;
- comentários, anexos e colaboração;
- temporizadores de SLA e escalonamento;
- resolução, causa, classificação e falso positivo;
- auditoria completa.

3.7 Catálogo de Eventos

A tela cataloga 20 tipos e apresenta contadores, categorias, severidades e tempos de resposta. Todo esse material é mock. Os únicos detectores comportamentais reais encontrados no motor atual são:

- aceleração brusca;
- frenagem brusca;
- curva brusca;
- excesso de velocidade.

Itens como pânico, entrada/saída de cerca, violação do rastreador, roubo, colisão, reboque, horário incomum, distração, fadiga, combustível e manutenção representam intenção de produto, não capacidade comprovada.

O catálogo deve deixar explícitos três estados: **disponível**, **planejado** e **indisponível para a telemetria atual**. Também deve informar quais fontes e sensores sustentam cada detector.

3.8 Mapa

O mapa funciona como pano de fundo em todas as áreas, porém ainda não é uma ferramenta operacional completa. Faltam:

- desenho e edição de círculo, polígono e corredor;
- camadas de cercas, rotas, risco, incidentes e cobertura;
- agrupamento de marcadores;
- cores por estado real do veículo;
- replay de viagem com linha do tempo;
- comparação entre rota planejada e realizada;
- seleção espacial e ações em lote;
- menu contextual no veículo;
- painel de atributos e telemetria;
- histórico de posições com qualidade e origem;
- indicação de dado atrasado, interpolado, simulado ou em replay.

---

4. Matriz de verdade da interface

Recurso exibido	Origem atual	Pode ser tratado como operacional?
Posições Tracker	`/api/posicoes`	Parcialmente
Usuários/rotas Dirija Bem	APIs Dirija Bem	Parcialmente
Scores e eventos comportamentais	Motor em memória	Parcialmente
Timeline	API real	Sim, após deduplicação
Análises	APIs reais	Parcialmente
Lista de monitores	API/banco	Parcialmente
Veículos no detalhe do monitor	Mistura API e mock	Não
Eventos no detalhe do monitor	Mock	Não
Alertas e análise de IA	Mock	Não
Ações reconhecer/resolver	Mock, sem persistência	Não
Catálogo e estatísticas de eventos	Mock	Não
Cercas e eventos de cerca	Apenas exemplos mock	Não
Comunicação e contato	Apenas narrativa mock	Não
Agente autônomo	Não existe runtime	Não

Regra imediata de UX

Até cada integração ficar pronta, toda informação simulada deve receber uma etiqueta visível **DEMONSTRAÇÃO**. Em modo operacional, mocks devem ser desabilitados. Erro de API deve aparecer como erro ou estado degradado; nunca deve ser substituído silenciosamente por uma informação plausível.

---

5. Arquitetura de produto proposta

A navegação deve ser reorganizada por trabalho operacional, e não somente pelos componentes atuais.

Menu principal proposto

1. **Central** — visão consolidada da operação;
2. **Mapa ao vivo** — veículos, cercas, rotas e incidentes;
3. **Incidentes** — fila de trabalho e atendimento;
4. **Frota** — veículos, motoristas, viagens, sensores e segurança;
5. **Cercas e rotas** — cadastro, atribuição e regras;
6. **Agentes de IA** — configuração, execução e auditoria;
7. **Eventos e políticas** — catálogo real, detectores e limites;
8. **Análises** — comportamento, segurança e operação;
9. **Integrações** — Tracker, Dirija Bem, mensageria, telefonia e saúde;
10. **Administração** — usuários, papéis, contatos e auditoria.

O mapa pode continuar visível em várias telas, mas não deve ocupar espaço quando o trabalho principal for configurar política, investigar incidente ou auditar uma ação.

---

6. Especificação das novas áreas

6.1 Central de comando

Deve responder em poucos segundos:

- quantos veículos estão online, atrasados e offline;
- quantos estão em movimento, parados, com ignição ligada e em alarme;
- incidentes críticos abertos, sem responsável e fora do SLA;
- cercas violadas;
- veículos possivelmente roubados;
- agentes saudáveis, degradados ou pausados;
- ações automáticas aguardando confirmação;
- falhas das fontes Tracker e Dirija Bem;
- mensagens e ligações sem resposta.

Widgets devem permitir abrir diretamente a lista filtrada. Um contador sem drill-down não é suficiente.

6.2 Frota e identidade unificada

Criar uma entidade unificada de veículo que relacione:

- identificador interno;
- `device_id` Tracker;
- `VEICOD`/identificador legado;
- usuário/motorista Dirija Bem;
- placa, chassi, marca, modelo, ano e grupo;
- proprietário, empresa e contatos;
- dispositivo instalado, protocolo e capacidades;
- fonte primária e fontes auxiliares;
- última posição e última comunicação de cada fonte.

A tela do veículo deve ter abas:

- Visão geral;
- Ao vivo;
- Viagens;
- Comportamento;
- Incidentes;
- Cercas e rotas;
- Sensores e comandos;
- Manutenção;
- Agentes responsáveis;
- Histórico e auditoria.

6.3 Cercas eletrônicas

Permitir criar no mapa:

- círculo;
- polígono;
- corredor de rota;
- ponto com raio;
- zona composta, quando necessário.

Cada cerca deve possuir nome, descrição, categoria, cor, empresa, vigência, fuso horário, agenda semanal, tolerância, veículos/grupos vinculados, regra de entrada, saída, permanência e tempo excedido.

Para evitar alarmes na borda, implementar histerese e permanência mínima. O evento deve registrar cerca, veículo, horário, posição, precisão GPS, direção do cruzamento e regra aplicada.

Casos principais:

- saída de garagem fora do horário;
- entrada em zona de risco;
- permanência indevida;
- saída de área autorizada;
- chegada/saída de cliente;
- proteção patrimonial em modo armado.

6.4 Rotas e desvios

Permitir cadastrar ou importar rota, atribuir veículo, motorista e janela de execução, definir corredor de tolerância, pontos obrigatórios, paradas e sequência.

Detectar:

- início e conclusão;
- desvio lateral;
- parada não planejada;
- ponto obrigatório ignorado;
- atraso;
- retorno inesperado;
- encerramento fora do destino.

6.5 Sensores, capacidades e comandos

A interface não deve usar um único botão genérico “habilitar sensor”. Existem três conceitos diferentes:

1. **Capacidade física:** o dispositivo possui ou não a entrada/sensor;
2. **Saúde da telemetria:** o sinal está chegando e é confiável;
3. **Política de detecção:** o sistema deve ou não gerar evento/alerta a partir daquele sinal.

Somente dispositivos com protocolo bidirecional comprovado podem receber comandos físicos. Para cada sensor, exibir disponibilidade, valor, unidade, última leitura, qualidade, origem, estado e política associada.

Sensores/capacidades desejados:

- GPS, GSM e ignição;
- alimentação externa e bateria interna;
- entrada de pânico;
- violação/tamper;
- velocidade e odômetro;
- acelerômetro;
- portas e baú;
- engate/reboque;
- temperatura;
- combustível;
- bloqueio, sirene e outras saídas, quando suportadas.

Comandos críticos, como bloqueio, precisam de permissão, política, dupla confirmação ou aprovação, registro de solicitante, protocolo de entrega, confirmação do dispositivo e mecanismo seguro de reversão. Nunca mostrar “executado” apenas porque a requisição foi enviada.

6.6 Incidentes

Evento é uma observação técnica. Incidente é uma situação que exige acompanhamento. Vários eventos devem poder formar um incidente.

Estados propostos:

`novo → triagem → em atendimento → aguardando resposta → escalado → resolvido → encerrado`

Cada incidente deve conter:

- severidade e risco atual;
- veículo, motorista, proprietário e agentes envolvidos;
- eventos e evidências correlacionados;
- posição, trajetória e cercas;
- hipótese e confiança;
- responsável humano e/ou agente;
- SLA e temporizadores;
- ações tentadas, resultados e próximos passos;
- conversas e gravações permitidas;
- motivo de resolução e classificação final.

6.7 Agentes de IA autônomos

Um monitor só se torna agente quando possui objetivo, percepção, memória, ferramentas, política de decisão e capacidade de acompanhar o resultado.

Perfil configurável

- nome e missão;
- fontes e veículos sob responsabilidade;
- eventos e incidentes que observa;
- horário de atuação;
- dados que pode consultar;
- ferramentas que pode usar;
- ações permitidas, proibidas e sujeitas a aprovação;
- contatos e ordem de escalonamento;
- limites de tentativas, custo e tempo;
- modelo de IA, versão do prompt e fallback;
- nível de autonomia.

Níveis de autonomia

Nível	Comportamento
0 — Observador	Analisa e registra, sem agir
1 — Recomendador	Propõe ações para aprovação humana
2 — Assistido	Executa ações de baixo risco e pede aprovação nas demais
3 — Autônomo limitado	Conduz o procedimento dentro de uma política publicada
4 — Emergência	Executa playbook especial, com limites e auditoria reforçados

O nível não pode transformar uma ação proibida em permitida. Bloqueio remoto, comunicação com autoridades e ações com risco físico devem possuir regras próprias.

Tela operacional do agente

Exibir:

- estado e saúde;
- missão e escopo;
- última e próxima execução;
- casos em análise;
- decisões recentes;
- ferramentas utilizadas;
- ações pendentes, concluídas e falhas;
- justificativa e evidências de cada decisão;
- memória usada;
- consumo, latência e custo;
- versão da política e do prompt;
- botão pausar, assumir manualmente e executar teste.

Como o agente deve agir como monitor humano

1. Perceber o contexto, não apenas um evento isolado;
2. Confirmar qualidade e atualidade dos dados;
3. Consultar veículo, motorista, cercas, rota, horário e histórico;
4. Formular hipóteses e indicar confiança;
5. Escolher o playbook compatível com risco e contrato;
6. Enviar mensagem ou iniciar ligação quando permitido;
7. Entender a resposta e atualizar a hipótese;
8. Tentar canal alternativo dentro do limite;
9. Escalar para responsável ou operador;
10. Acompanhar até normalização ou encerramento;
11. Registrar tudo em linguagem auditável;
12. Aprender com a classificação final sem alterar políticas sozinho.

6.8 Comunicações e ações

Criar uma fila persistente de ações, com idempotência e retentativas. Tipos iniciais:

- notificação interna;
- WhatsApp;
- SMS;
- e-mail;
- ligação automatizada;
- webhook;
- tarefa para operador;
- solicitação de aprovação;
- comando para dispositivo, quando suportado.

Estados mínimos:

`planejada → aguardando aprovação → enfileirada → enviada → entregue → respondida → concluída`

Também devem existir `falhou`, `cancelada`, `expirada` e `não suportada`.

Cada mensagem precisa registrar destinatário, canal, template, conteúdo final, motivo, agente, incidente, fornecedor, tentativas e recibos. A IA pode adaptar texto e conduzir diálogo, mas fatos essenciais e instruções de segurança devem vir de templates/políticas controladas.

6.9 Integrações e saúde

Painel específico para:

- socket Tracker e protocolos;
- leitura Dirija Bem;
- atraso por fonte;
- volume e erros;
- último pacote recebido;
- filas e workers;
- banco de dados;
- provedor de IA;
- WhatsApp/SMS/e-mail/telefonia;
- comandos de dispositivos;
- custos e limites.

Cada fonte deve informar `online`, `degradada`, `offline` ou `em manutenção`. Replay e simulador devem ser explicitamente identificados.

---

7. Jornadas operacionais prioritárias

7.1 Possível roubo

1. Veículo armado sai da cerca fora da agenda;
2. motor correlaciona ignição, movimento, horário, rota e contato;
3. abre um incidente único;
4. agente valida frescor e qualidade dos sinais;
5. envia pergunta segura ao proprietário;
6. se não houver resposta, tenta canal alternativo;
7. se o proprietário negar autorização, eleva severidade;
8. oferece ao operador ações permitidas pelo contrato;
9. acompanha deslocamento e atualiza o incidente;
10. encerra somente após confirmação e registra evidências.

7.2 Direção perigosa

1. Eventos são agrupados por viagem e exposição;
2. agente consulta dados Tracker e Dirija Bem;
3. distingue episódio isolado de padrão recorrente;
4. em risco imediato, envia orientação curta e segura conforme política;

5. para recorrência, agenda contato ou relatório ao gestor;
6. acompanha melhora e registra resultado.

7.3 Veículo offline

1. Detecta atraso acima do limite específico do dispositivo;
2. verifica manutenção, cobertura e falha geral da fonte;
3. evita abrir centenas de incidentes para uma indisponibilidade sistêmica;
4. tenta diagnóstico e contato conforme criticidade;
5. escala se veículo protegido estiver em contexto de risco;
6. fecha após retomada estável e período de observação.

7.4 Violação de cerca

1. Aplica agenda, tolerância, precisão e histerese;
2. agrega cruzamentos oscilantes em um episódio;
3. verifica rota autorizada e ordem de serviço;
4. abre ou atualiza incidente;
5. agente executa o playbook e acompanha retorno à condição segura.

---

8. Requisitos transversais

Segurança e privacidade

- autenticação obrigatória;
- separação por empresa/cliente;
- papéis e permissões por ação;
- mascaramento de dados pessoais conforme perfil;
- trilha de auditoria imutável;
- segredos fora do frontend e do repositório;
- confirmação reforçada para ações críticas;
- política de retenção de localização, mensagens e gravações;
- consentimento e base legal para comunicações.

Confiabilidade

- persistir eventos, scores, incidentes, decisões e ações;
- não depender de estado em memória para operação;
- idempotência e deduplicação;
- filas com retentativa e fila de falhas;
- métricas, logs estruturados e alertas técnicos;
- modo degradado explícito;
- nenhuma substituição silenciosa por mock.

Usabilidade e acessibilidade

- estados não podem depender somente de cor;
- navegação por teclado e foco visível;
- contraste e textos legíveis;
- horário com fuso explícito;
- design responsivo para notebook e segundo monitor;
- atalhos para assumir, reconhecer, escalar e abrir o veículo;
- confirmação proporcional ao risco da ação.

---

9. Contratos de frontend necessários

O frontend deve usar um único cliente de API configurável, eliminando URLs `localhost:5009` espalhadas nos componentes.

Recursos mínimos:

- `/api/v1/vehicles` e `/api/v1/vehicles/{id}`;
- `/api/v1/positions` e histórico;
- `/api/v1/trips`;
- `/api/v1/geofences` e atribuições;
- `/api/v1/routes` e execuções;
- `/api/v1/sensors` e políticas;
- `/api/v1/events` e episódios;
- `/api/v1/incidents` e timeline;
- `/api/v1/agents`, versões, execuções e decisões;
- `/api/v1/actions`, aprovações e recibos;
- `/api/v1/contacts` e canais;
- `/api/v1/integrations/health`;
- `/api/v1/audit`.

As respostas devem carregar origem, horário da fonte, horário de recebimento, qualidade, modo (`live`, `replay`, `simulation`) e identificador de correlação.

---

10. Plano de implementação incremental

Cada item abaixo foi delimitado para caber em até 30 minutos de execução focada. Ao tocar código de produção, criar/ajustar testes e executá-los antes do commit.

Etapa 0 — Verdade e segurança da interface

- [ ] **0.1** Criar indicador global de ambiente: simulação, replay ou produção.
- [ ] **0.2** Etiquetar todas as seções alimentadas por mock como DEMONSTRAÇÃO.
- [ ] **0.3** Remover fallback silencioso de score/veículos no detalhe do monitor.

- [ ] **0.4** Criar estado de erro, vazio e carregamento comum.
- [ ] **0.5** Centralizar base URL e cliente HTTP em `apiService.ts`.
- [ ] **0.6** Criar teste que falhe se componente novo usar URL hardcoded.
- [ ] **0.7** Documentar matriz tela → endpoint → fonte → modo.
- [ ] **0.8** Separar visualmente Tracker e Dirija Bem sem perder visão unificada.

Aceite: nenhum dado de demonstração pode ser confundido com informação operacional.

Etapa 1 — Estabilizar eventos e identidade

- [ ] **1.1** Definir DTO de veículo unificado.
- [ ] **1.2** Expor origem, modo e última comunicação no DTO.
- [ ] **1.3** Criar lista unificada pesquisável.
- [ ] **1.4** Criar cabeçalho de detalhe do veículo.
- [ ] **1.5** Implementar episódio de excesso de velocidade.
- [ ] **1.6** Implementar cooldown/histerese dos demais detectores.
- [ ] **1.7** Persistir eventos e scores essenciais.
- [ ] **1.8** Corrigir contadores e scores para usar episódios.
- [ ] **1.9** Adicionar filtros à timeline.
- [ ] **1.10** Adicionar drill-down evento → posição/veículo.

Aceite: condição contínua gera um episódio, e Tracker/Dirija Bem aparecem com identidade e origem inequívocas.

Etapa 2 — Incidentes reais

- [ ] **2.1** Criar migration de incidentes.
- [ ] **2.2** Criar migration de timeline do incidente.
- [ ] **2.3** Criar API de listagem e detalhe.
- [ ] **2.4** Criar máquina de estados e validar transições.
- [ ] **2.5** Correlacionar episódios ao incidente.
- [ ] **2.6** Substituir Alertas mock por fila real de incidentes.
- [ ] **2.7** Implementar reconhecer, assumir e comentar.
- [ ] **2.8** Implementar resolver e classificar falso positivo.
- [ ] **2.9** Adicionar SLA e escalonamento visual.
- [ ] **2.10** Criar testes de concorrência e idempotência.

Aceite: operador conduz um caso completo, persistente e auditável, sem dados mock.

Etapa 3 — Cercas eletrônicas

- [ ] **3.1** Criar migration de cercas e versões geométricas.
- [ ] **3.2** Criar migration de vínculos e agendas.
- [ ] **3.3** Criar CRUD de cercas.
- [ ] **3.4** Adicionar desenho de círculo no mapa.
- [ ] **3.5** Adicionar desenho de polígono.
- [ ] **3.6** Adicionar edição, validação e exclusão segura.
- [ ] **3.7** Criar tela de atribuição a veículos/grupos.

- [ ] **3.8** Implementar entrada/saída com histerese.
- [ ] **3.9** Implementar agenda e permanência.
- [ ] **3.10** Integrar episódios de cerca aos incidentes.
- [ ] **3.11** Criar replay de teste de cruzamento.
- [ ] **3.12** Mostrar cercas e estado no mapa ao vivo.

Aceite: uma cerca criada visualmente detecta entrada/saída real ou simulada sem oscilar na borda.

Etapa 4 — Sensores e segurança patrimonial

- [ ] **4.1** Catalogar capacidades por protocolo/dispositivo.
- [ ] **4.2** Criar modelo de sensor e leitura.
- [ ] **4.3** Criar modelo de política de detector.
- [ ] **4.4** Criar aba Sensores do veículo.
- [ ] **4.5** Exibir valor, saúde, origem e última leitura.
- [ ] **4.6** Implementar ativação de política, não de hardware.
- [ ] **4.7** Criar estados armado/desarmado.
- [ ] **4.8** Detectar alimentação removida e bateria baixa quando suportado.
- [ ] **4.9** Detectar offline contextualizado.
- [ ] **4.10** Definir contrato de comando e confirmação.
- [ ] **4.11** Criar autorização reforçada para comandos críticos.
- [ ] **4.12** Criar simulador de sinais suportados.

Aceite: operador sabe o que o dispositivo suporta, o que está saudável e quais detecções estão habilitadas.

Etapa 5 — Runtime dos agentes

- [ ] **5.1** Criar migration de versões/configuração do agente.
- [ ] **5.2** Criar migration de execuções e decisões.
- [ ] **5.3** Definir esquema estruturado da decisão da IA.
- [ ] **5.4** Implementar provedor de IA atrás de uma interface.
- [ ] **5.5** Implementar timeout, fallback e circuit breaker.
- [ ] **5.6** Criar ferramentas somente leitura: veículo, posição, histórico e cerca.
- [ ] **5.7** Criar política de ferramentas por agente.
- [ ] **5.8** Criar níveis de autonomia.
- [ ] **5.9** Criar worker de análise de incidente.
- [ ] **5.10** Persistir evidências, decisão e confiança.
- [ ] **5.11** Criar tela de configuração do agente.
- [ ] **5.12** Criar tela de execução/decisões.
- [ ] **5.13** Implementar pausar e assumir manualmente.
- [ ] **5.14** Criar modo simulação sem ações externas.
- [ ] **5.15** Criar avaliação com casos esperados.

Aceite: agente analisa incidente com contexto real, produz decisão estruturada, auditável e não executa ferramenta fora da política.

Etapa 6 — Ações e comunicação

- [ ] **6.1** Criar migration da fila de ações.
- [ ] **6.2** Implementar máquina de estados e idempotência.
- [ ] **6.3** Criar ferramenta de notificação interna.
- [ ] **6.4** Integrar primeiro canal de mensagem em sandbox.
- [ ] **6.5** Persistir entrega, leitura e resposta.
- [ ] **6.6** Criar templates versionados.
- [ ] **6.7** Criar contatos e ordem de escalonamento.
- [ ] **6.8** Criar solicitação/aprovação humana.
- [ ] **6.9** Criar timeline de ações no incidente.
- [ ] **6.10** Implementar retentativas e fila de falhas.
- [ ] **6.11** Integrar canal de voz em sandbox.
- [ ] **6.12** Transcrever/classificar resposta conforme política.
- [ ] **6.13** Criar limites de tentativa, horário e custo.
- [ ] **6.14** Testar opt-out, falha e ausência de resposta.

Aceite: uma ação percorre toda a fila, gera recibo real e aparece no incidente com responsável e resultado.

Etapa 7 — Rotas, análises e gestão

- [ ] **7.1** Criar CRUD de rotas e corredor.
- [ ] **7.2** Criar atribuição e janela de execução.
- [ ] **7.3** Detectar desvio e parada não planejada.
- [ ] **7.4** Criar replay de viagem.
- [ ] **7.5** Adicionar período e comparação às análises.
- [ ] **7.6** Normalizar comportamento por distância/tempo.
- [ ] **7.7** Criar relatórios por Tracker e Dirija Bem.
- [ ] **7.8** Criar indicadores de SLA e automação.
- [ ] **7.9** Criar exportação e agendamento.
- [ ] **7.10** Validar score com amostra conhecida.

Aceite: gestão consegue explicar tendência e score, e rastrear qualquer métrica até a evidência.

Etapa 8 — Produção, governança e UX

- [ ] **8.1** Implementar autenticação.
- [ ] **8.2** Implementar papéis e permissões.
- [ ] **8.3** Implementar isolamento por empresa.
- [ ] **8.4** Criar trilha de auditoria consultável.
- [ ] **8.5** Criar painel de integrações e saúde.
- [ ] **8.6** Adicionar métricas, logs e alertas técnicos.
- [ ] **8.7** Executar teste de carga e recuperação.
- [ ] **8.8** Executar auditoria de segurança e privacidade.
- [ ] **8.9** Executar teste de acessibilidade.
- [ ] **8.10** Executar teste operacional com playbooks.

- [ ] **8.11** Publicar runbook e plano de contingência.
- [ ] **8.12** Liberar produção por grupo piloto.

Aceite: operação real possui acesso controlado, observabilidade, contingência e piloto aprovado.

---

11. Prioridades recomendadas

P0 — Fazer antes de qualquer automação externa

1. Identificar claramente mocks, simulação e replay;
2. remover fallback silencioso;
3. agrupar eventos em episódios;
4. persistir estado operacional;
5. unificar identidade Tracker/Dirija Bem;
6. criar incidentes reais;
7. adicionar autenticação e auditoria mínima.

P1 — Formar a central operacional

1. Cercas eletrônicas;
2. sensores/capacidades;
3. políticas de segurança;
4. configuração e observabilidade dos agentes;
5. fila de ações e primeiro canal de mensagem;
6. contatos, aprovações e escalonamentos.

P2 — Ampliar inteligência e eficiência

1. Telefonia;
2. rotas e desvios;
3. comandos bidirecionais suportados;
4. relatórios avançados;
5. otimização de custos e avaliação contínua da IA.

---

12. Critérios para chamar o sistema de “monitor autônomo”

O produto só deve ser apresentado como autônomo quando demonstrar, em ambiente controlado:

- percepção de eventos reais de ambas as fontes;
- correlação em incidentes sem tempestade de alertas;
- decisão baseada em evidências e política versionada;
- ação real por ao menos um canal;

- interpretação da resposta;
- acompanhamento até desfecho;
- escalonamento quando necessário;
- limites de autonomia respeitados;
- intervenção humana imediata;
- trilha completa de dados, raciocínio, ferramenta, ação e resultado;
- comportamento seguro quando IA, banco, canal ou telemetria falhar.

Até lá, o termo correto é **protótipo de monitoramento com componentes de automação planejados**.

---

13. Conclusão

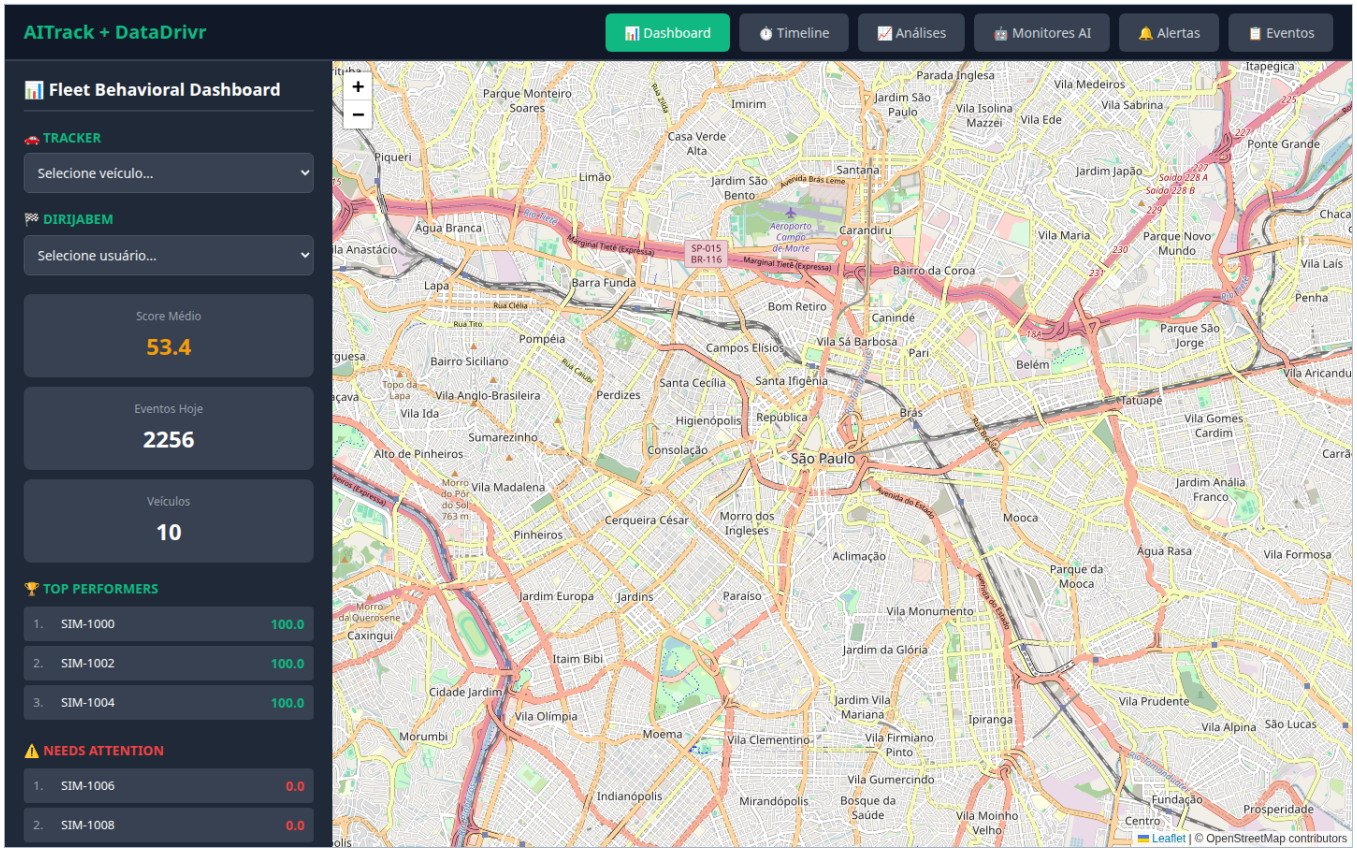
O sistema capturou corretamente os três pilares iniciais do produto: duas fontes de rastreamento/comportamento, mapa operacional e conceito de monitores vinculados a veículos. A interface atual é útil para demonstrar a visão, mas ainda mascara lacunas com mocks e não permite que um agente substitua o trabalho de um monitor humano.

O caminho mais seguro não é começar pela ligação automática. Primeiro é necessário garantir verdade dos dados, episódios, identidade unificada e incidentes persistentes. Em seguida entram cercas, sensores e políticas. Só então a IA deve receber ferramentas e autonomia progressiva. A comunicação externa vem por uma fila auditável, com confirmações e limites.

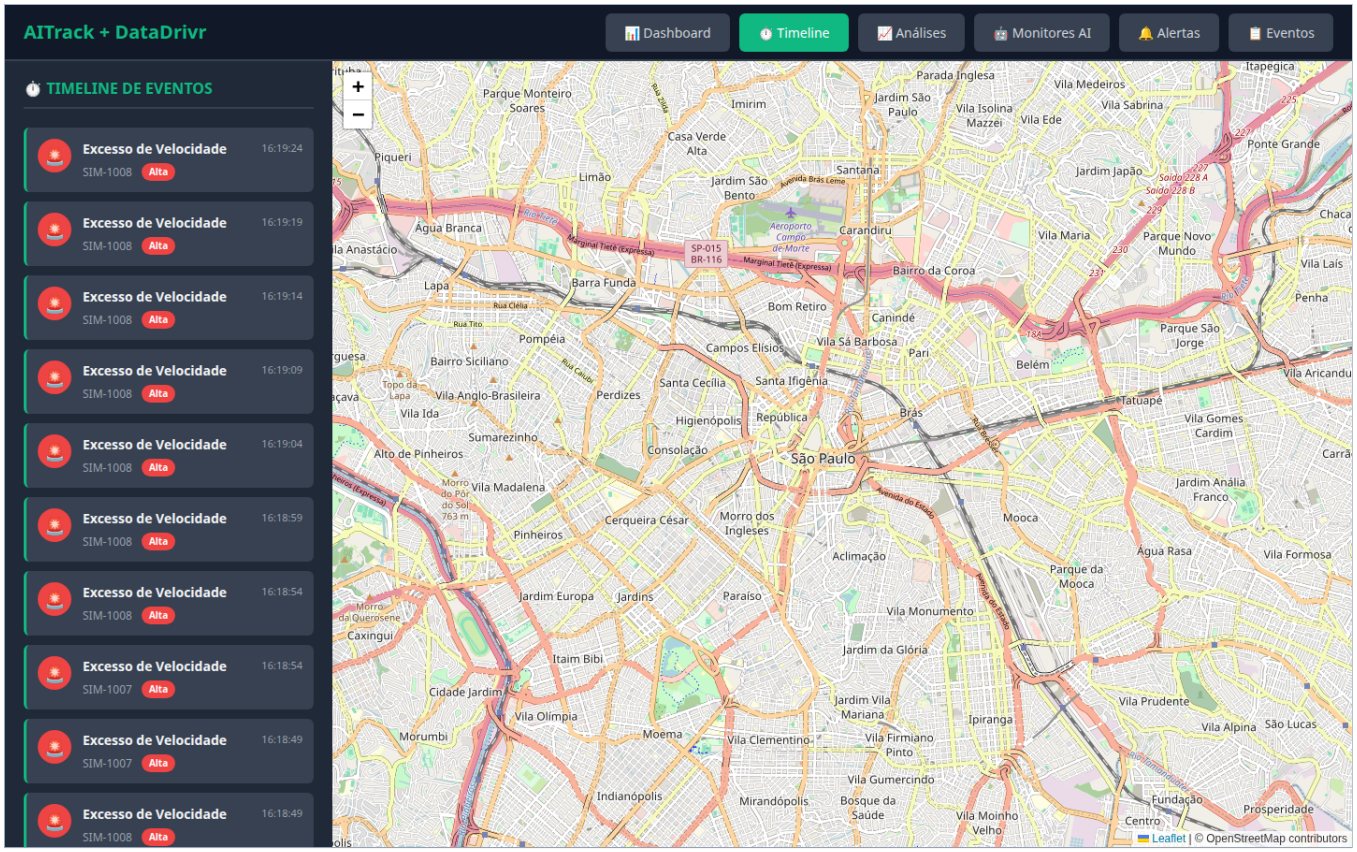
Esse encadeamento transforma o protótipo atual em uma central real sem criar uma automação rápida, convincente e perigosa.

Evidências visuais da auditoria

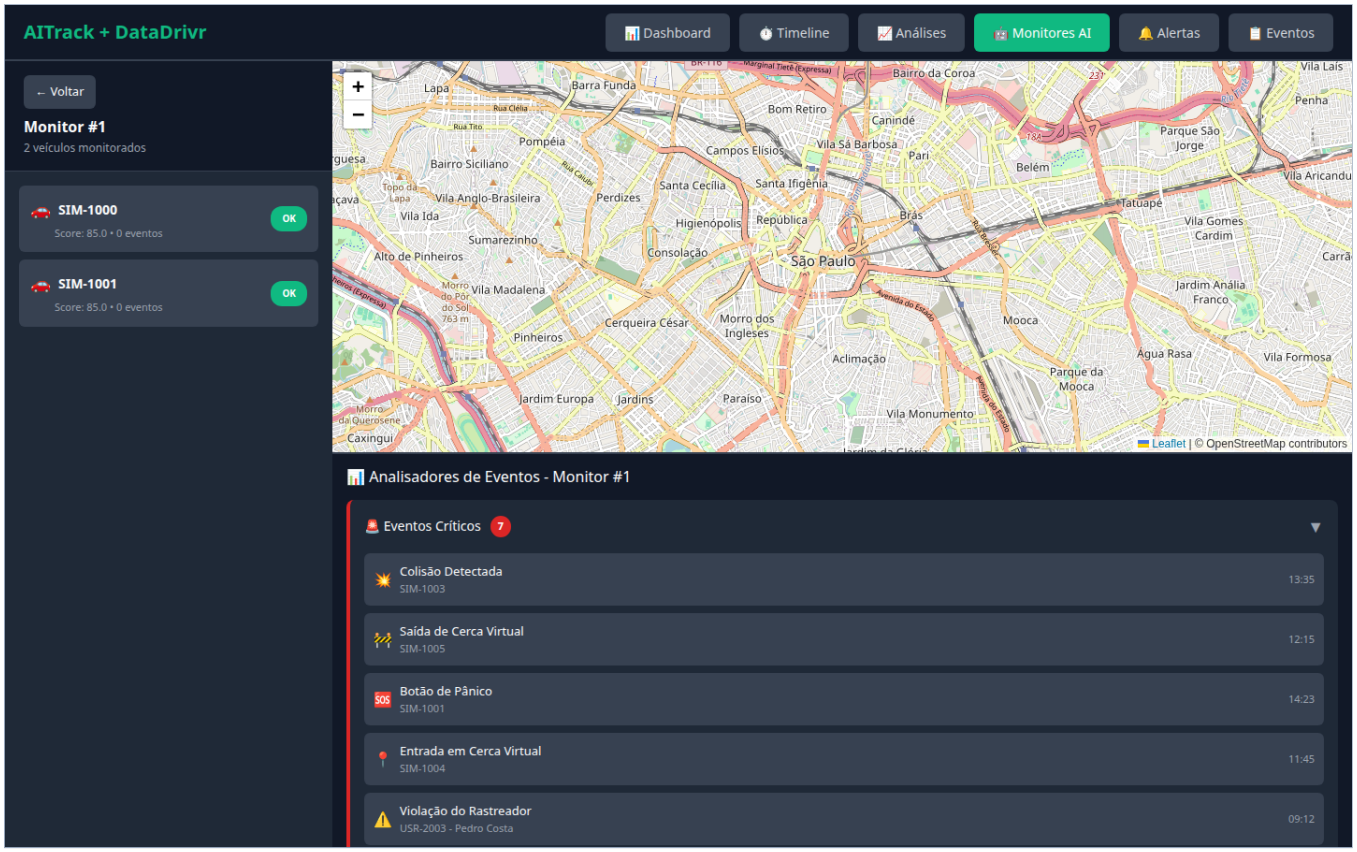
Capturas produzidas automaticamente durante a navegação Playwright em 05/11/2026.



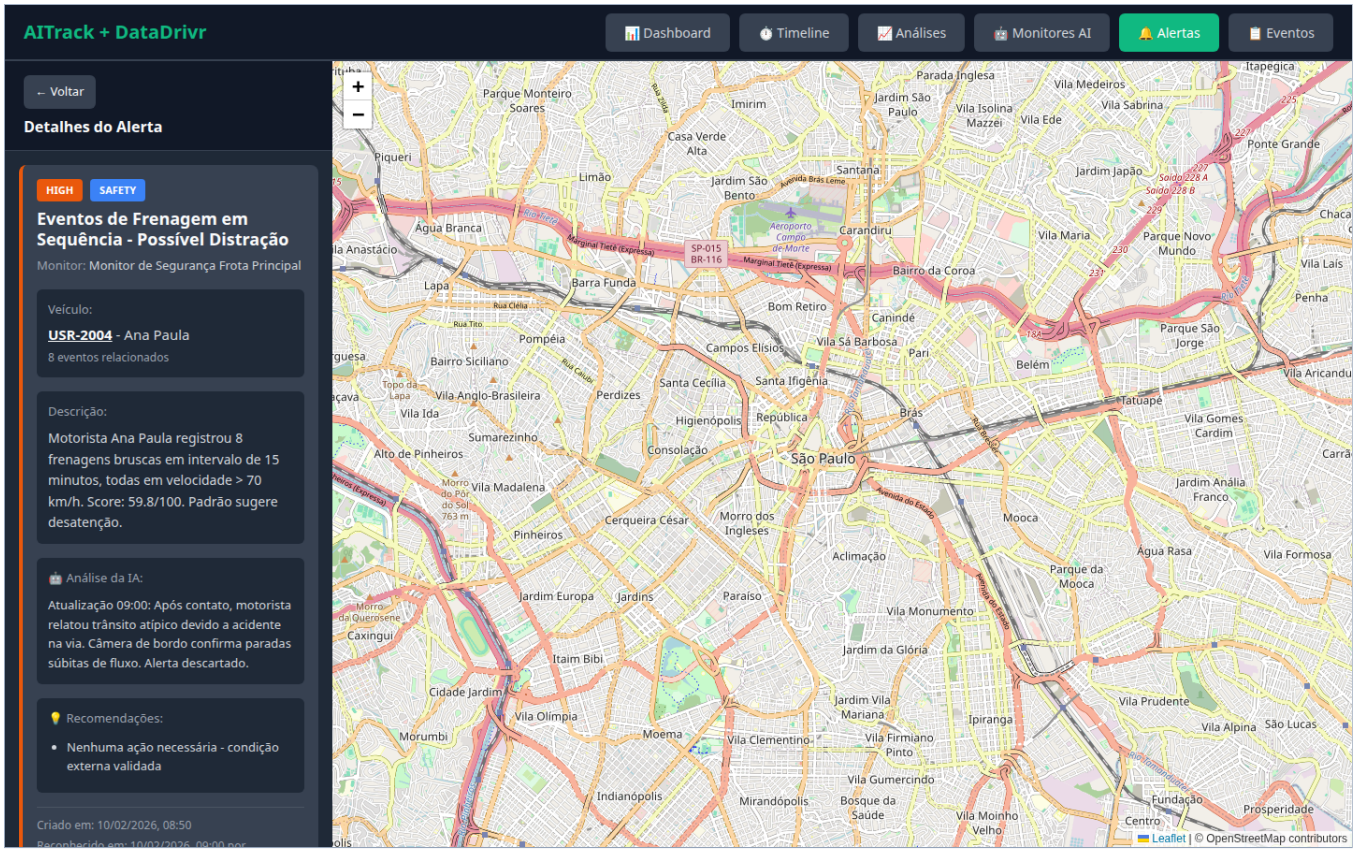
Dashboard com dados reais de frota e mapa.



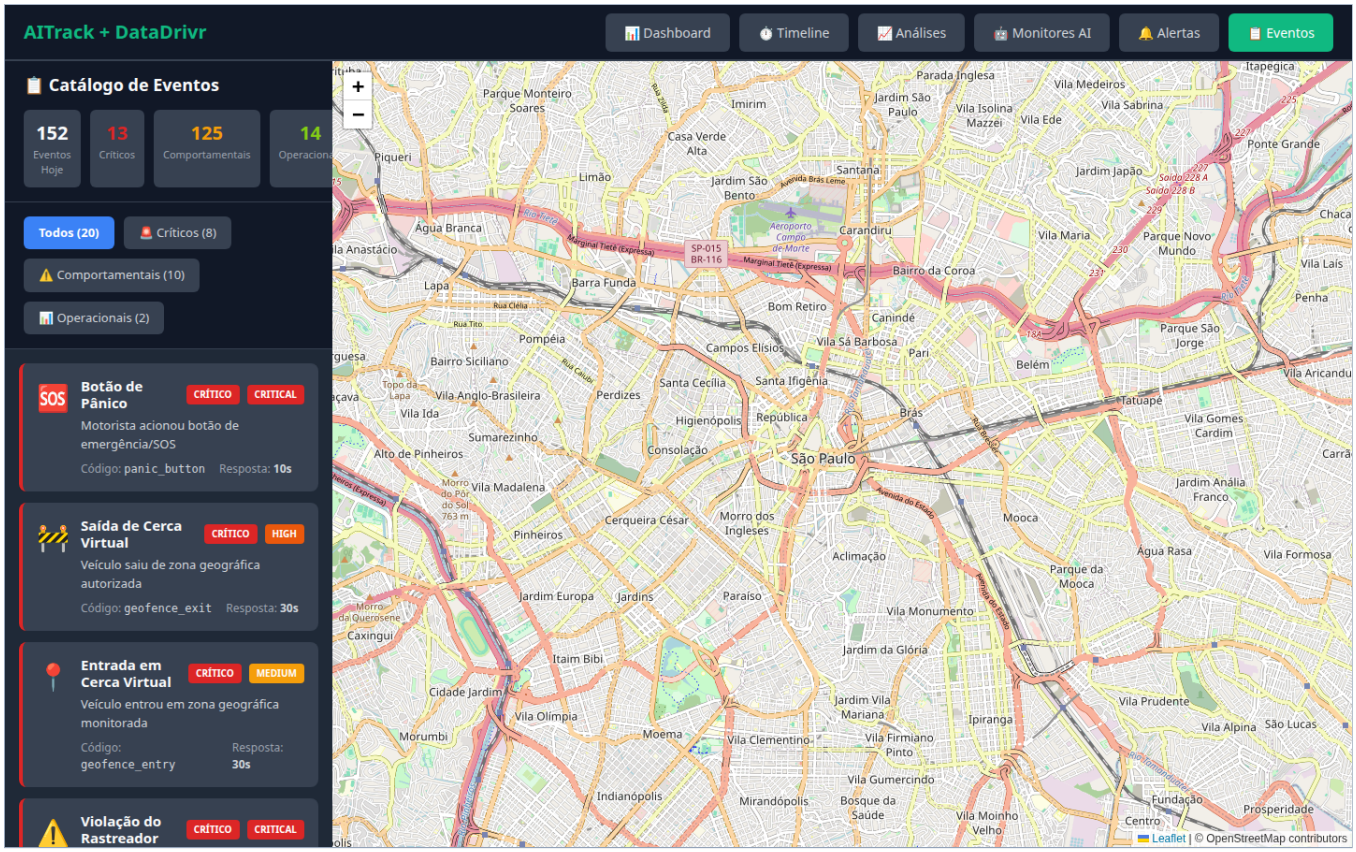
Timeline real evidenciando repetição de excesso de velocidade.



Detalhe do monitor: veículos e eventos apresentados em conjunto, com eventos mock.



Detalhe de alerta demonstrativo apresentado como análise de IA.



Catálogo demonstrativo com capacidades ainda não implementadas.